Introduction to Databases ? Lecture 3: Query Processing and Indexing

Database management systems (DBMS) are software systems that allow users to define, create, maintain, and control access to data. A relational DBMS organises data into tables (relations) with rows (tuples) and columns (attributes).

SQL (Structured Query Language) is the standard language for querying relational databases. A simple SELECT statement retrieves rows from a table that match a WHERE condition.

Query processing involves translating a high-level SQL query into an efficient sequence of low-level operations. The main stages are: parsing and translation, optimisation, and evaluation.

The query parser checks syntax and converts the SQL statement into a relational algebra expression. The query optimiser then selects the most efficient execution plan from among many equivalent plans.

Relational algebra is the formal language underlying SQL. Key operators include: Selection (?) ? filters rows; Projection (?) ? selects columns; Join (?) ? combines rows from two relations; and Aggregation ? computes group-level statistics.

Query optimisation exploits algebraic equivalences, such as pushing selections down the query tree to reduce the number of tuples processed early.

Indexes are auxiliary data structures that speed up data retrieval. Without an index, a query must perform a full table scan ? reading every row to find matching tuples, which costs O(n) I/O operations.

A B-tree index organises keys in a balanced tree structure, allowing equality and range lookups in O(log n) time. Most relational databases use B+ trees, where all data pointers reside in the leaf nodes.

A hash index uses a hash function to map keys to buckets. Hash indexes support equality lookups in O(1) average time but cannot support range queries.

Clustered indexes store the table rows in the same order as the index keys. A table can have at most one clustered index. Non-clustered indexes store pointers to the actual row location.

Join algorithms are critical for query performance. The nested-loop join is the simplest: for each tuple in the outer relation, scan the entire inner relation for matching tuples. Its cost is O(n × m).

The merge join requires both relations to be sorted on the join attribute, then merges them in one pass ? O(n + m) for sorted inputs. The hash join builds an in-memory hash table on the smaller relation, then probes it for each tuple of the larger relation.

Transaction management ensures data integrity when multiple users access the database concurrently. A transaction is a sequence of operations that must be executed atomically ? either all succeed or all are rolled back.

The ACID properties define correct transaction behaviour: Atomicity, Consistency, Isolation, and Durability. Isolation is implemented using locking protocols or multiversion concurrency control (MVCC).

Normalisation is the process of organising a relational schema to reduce data redundancy and update anomalies. The first normal form (1NF) requires atomic attribute values. Second normal form (2NF) eliminates partial dependencies. Third normal form (3NF) eliminates transitive dependencies.

Boyce-Codd normal form (BCNF) is a stricter version of 3NF: for every functional dependency X ? Y, X must be a superkey.

NoSQL databases depart from the relational model to address specific scalability and flexibility requirements. Document stores (e.g. MongoDB) store JSON-like documents. Key-value stores (e.g. Redis) provide O(1) lookups by key. Column-family stores (e.g. Cassandra) are optimised for write-heavy workloads.

The CAP theorem states that a distributed system can guarantee at most two of: Consistency, Availability, and Partition tolerance.

Summary: We covered query processing stages, relational algebra, index structures (B-tree, hash, clustered), join algorithms, ACID transactions, normalisation, and an overview of NoSQL systems. Next lecture: concurrency control and recovery.